02 / PYTHON REFERENCE GUIDE

NumPy tools for 3D rotations

EX30 · Inputs, outputs, examples and common errors

Objective. This guide describes the NumPy tools needed to construct and verify 3D rotation matrices. Every entry explains its inputs, outputs, a small example and common mistakes. This is not a full EX30 solution.

1. `np.deg2rad()`: angle conversion

Input: a number or array of angles in degrees. **Output:** the equivalent radians, as a scalar or array. NumPy trigonometric functions expect radians.

```
angle_deg = 90.0
angle_rad = np.deg2rad(angle_deg)
# angle_rad ~ 1.5707963268
```

Common error: calling `np.cos(90)` as if it represented $\cos 90°$. That evaluates cosine at 90 radians instead.

2. `np.sin()`, `np.cos()`: trigonometric inputs

Input: a scalar or array of radians. **Output:** numeric sine/cosine values of the same shape.

```
c = np.cos(np.deg2rad(90))  # approx. 0
s = np.sin(np.deg2rad(90))  # approx. 1
```

Floating-point note: $\cos 90°$ may print as 6.12×10^{-17}, which is effectively zero at normal numerical precision.

3. `np.array()`: constructing a 3 by 3 matrix

Input: a sequence of three rows, each containing three numbers. **Output:** an `ndarray` with shape `(3, 3)`. Here is a separate construction example using an angle α:

```
import numpy as np
alpha = np.deg2rad(45.0)
c = np.cos(alpha)
s = np.sin(alpha)
Rz = np.array([
    [c, -s, 0.0],
    [s,  c, 0.0],
    [0.0, 0.0, 1.0]
])
print(Rz.shape)  # (3, 3)
```

Programming rule: whenever the angle changes, compute c, s again and **rebuild** the matrix; changing only the angle variable does not update an existing array.

4. `@`: matrix-vector and matrix-matrix products

Input: dimensionally compatible arrays. **Output:** their matrix product. A `(3, 3)` matrix times a `(3,)` vector produces another `(3,)` vector.

```
I = np.eye(3)
v = np.array([2.0, -1.0, 4.0])
w = I @ v            # [2., -1., 4.]
print(w.shape)       # (3,)
```

Remember: `R2 @ R1 @ v` applies `R1` first and then `R2`. `R1 * R2` performs *elementwise multiplication*, not composition of rotations.

5. `.T` and `np.eye(3)`: transpose and identity

`.T`: an array property that returns a transposed *view*; on a 3-by-3 array it interchanges rows and columns.

`np.eye(3)`: takes the integer size 3 and returns the identity $\mathbf{I}_3$, with ones on the main diagonal and zeros elsewhere.

```
I = np.eye(3)
check_matrix = Rz.T @ Rz
print(np.allclose(check_matrix, I))  # True
```

For a proper rotation, $R_z^T R_z \simeq I_3$, and `Rz.T` undoes the rotation.

6. `np.allclose()` versus `np.isclose()`

`np.allclose(a,b)`: accepts two numeric arrays and returns **one boolean** after tolerance-based comparison.
`np.isclose(a,b)`: returns elementwise booleans.

```
a = np.array([1.0, 2.0, 3.0])
b = np.array([1.0, 2.0 + 1e-12, 3.0])
print(np.allclose(a, b))  # True
print(np.isclose(a, b))   # [ True  True  True]
```

Why it matters: it avoids exact equality checks on floating-point trigonometric results.

7. `np.linalg.norm()`, `np.linalg.det()`, `np.cross()`

Norm: `np.linalg.norm(v)` takes a vector and returns its Euclidean norm. For example, `np.linalg.norm([3,4,0])` returns 5.

Determinant: `np.linalg.det(R)` takes a square matrix and returns a scalar. A proper 3D rotation is expected to have a determinant of approximately $+1$.

Cross product: `np.cross(r, F)` takes two 3D vectors and returns a perpendicular vector, such as torque $\boldsymbol{M} = \boldsymbol{r} \times \boldsymbol{F}$ (EX27).

```
r = np.array([0.2, 0.0, 0.0])  # m
F = np.array([0.0, 0.0, 10.0]) # N
M = np.cross(r, F)             # [ 0., -2., 0.] N*m
```

Important: the check $\det R \simeq 1$ does not replace $R^T R \simeq I$.

8. Pre-run checklist

1. All angles are computed in radians, and new matrices are built for changed angles.
2. Every rotation matrix has shape `(3, 3)` and each vector has shape `(3,)`.
3. Correct R_x, R_y, R_z signs and `@` multiplication order are respected.
4. Results and norms are compared against manual calculations using numerical tolerances.
5. Units are printed for forces (N), lever arms (m) and torques (N m).